Connect Google Drive

```
from google.colab import drive
import os
import h5py
import pandas as pd

drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

A. Replication - Original Dataset

A-1. Import Libraries

```
!pip install pycox torchtuples h5py
```

```
!pip install lifelines
```

```
!pip install --upgrade scikit-survival
```

```
import h5py
import torch
import numpy as np
import pandas as pd
import os
import torch.nn as nn
from pycox.models import CoxPH
import torchtuples as tt
from pycox.evaluation import EvalSurv
from lifelines import CoxPHFitter
from sksurv.ensemble import RandomSurvivalForest
from sksurv.metrics import concordance_index_censored
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

A-2. Load Original Data

```
np.random.seed(48764388)
_ = torch.manual_seed(48764388)
```

```
# ================================================================
# Load the author's .h5 data file from Google Drive
# - metabric_IHC4_clinical_train_test.h5
# ================================================================
```

```
file_path = '/content/drive/MyDrive/metabric_IHC4_clinical_train_test.h5'

# Check if the file exists
if os.path.exists(file_path):
    with h5py.File(file_path, 'r') as f:
        # Extract Train / Test datasets
        x_train = f['train']['x'][:].astype('float32')
        e_train = f['train']['e'][:].astype('int32')
        t_train = f['train']['t'][:].astype('float32')

        x_test = f['test']['x'][:].astype('float32')
        e_test = f['test']['e'][:].astype('int32')
        t_test = f['test']['t'][:].astype('float32')

    print(".h5 data file successfully loaded!")
    print(f"- Number of train patients: {x_train.shape[0]} / Number of input features: {x_train.shape[1]}")
    print(f"- Number of test patients: {x_test.shape[0]}")

    # Feature names matching Table 1 and dataset description
    column_names = [
        'MKI67',
        'EGFR',
        'PGR',
        'ERBB2',
        'Hormone',
        'Radio',
        'Chemo',
        'ER-pos',
        'Age'
    ]

    # Create a pandas DataFrame for train data inspection
    df_train = pd.DataFrame(x_train, columns=column_names)
    df_train['Event (e)'] = e_train
    df_train['Time (t)'] = t_train

    # Inspect the top 5 patient records
    print("Wn Top 5 samples of Train data:")
    display(df_train.head())
else:
    print(f"File '{file_path}' not found. Please verify the file path.")
```

```
.h5 data file successfully loaded!
- Number of train patients: 1523 / Number of input features: 9
- Number of test patients: 381
```

Top 5 samples of Train data:

	MKI67	EGFR	PGR	ERBB2	Hormone	Radio	Chemo	ER-pos	Age	Event (e)	Time (t)
0	5.603834	7.811392	10.797988	5.967607	1.0	1.0	0.0	1.0	56.840000	0	99.333336
1	5.284882	9.581043	10.204620	5.664970	1.0	0.0	0.0	1.0	85.940002	1	95.733330
2	5.920251	6.776564	12.431715	5.873857	0.0	1.0	0.0	1.0	48.439999	0	140.233337
3	6.654017	5.341846	8.646379	5.655888	0.0	0.0	0.0	0.0	66.910004	0	239.300003
4	5.456747	5.339741	10.555724	6.008429	1.0	0.0	0.0	1.0	67.849998	1	56.933334

```
# Format target variables as (duration, event) tuples
y_train = (t_train, e_train)
y_test = (t_test, e_test)

# Split validation set (20% of the training data)
val_size = int(len(x_train) * 0.2)
x_val = x_train[:val_size]
y_val = (t_train[:val_size], e_train[:val_size])

x_train_sub = x_train[val_size:]
y_train_sub = (t_train[val_size:], e_train[val_size:])
```

A-3. Model Development (DeepSurv)

```
# ================================================================
# Build DeepSurv Neural Network Architecture (Tuned)
# ================================================================
in_features = x_train.shape[1]   # Number of input features (9)
num_nodes = [32, 32]             # Hidden layers
out_features = 1                 # Risk score output
dropout = 0.1

# Apply SELU activation function to enhance neural network performance
net = tt.practical.MLPVanilla(
    in_features,
    num_nodes,
    out_features,
    batch_norm=True,
    dropout=dropout,
    activation=nn.SELU # Apply SELU activation function
)

# Adjust learning rate (0.01 -> 0.001) for finer optimization
optimizer = tt.optim.Adam(lr=0.001, weight_decay=1e-4)
model = CoxPH(net, optimizer)

print("Model initialized with tuned hyperparameters!")
```

```
Model initialized with tuned hyperparameters!
```

```
# ================================================================
# DeepSurv Model Training
# ================================================================
batch_size = 256
epochs = 100
callbacks = [tt.callbacks.EarlyStopping(patience=10)]

print("Wn Starting DeepSurv model training...")
log = model.fit(
    x_train_sub,
    y_train_sub,
    batch_size=batch_size,
    epochs=epochs,
    callbacks=callbacks,
    val_data=(x_val, y_val),
    verbose=True
)
```

```
# ================================================================
# [Key Result] Evaluate and Display Test C-index (Concordance Index)
# ================================================================
# Predict survival probabilities/hazards on the test dataset
_ = model.compute_baseline_hazards()
surv = model.predict_surv_df(x_test)

# Calculate C-index using the survival evaluation class (EvalSurv)
ev = EvalSurv(surv, t_test, e_test, censor_surv='km')
c_index = ev.concordance_td()

print(f"[Replication Result] DeepSurv Test C-index: {c_index:.4f}")
```

```
[Replication Result] DeepSurv Test C-index: 0.6260
```

A-4. Model Development (CPH and RSF)

```
!pip install -q lifelines scikit-survival
```

```
import numpy as np
import pandas as pd
from lifelines import CoxPHFitter
from sksurv.ensemble import RandomSurvivalForest

# ================================================================
# 1. Align and Train Cox Proportional Hazards (CPH) Model
# ================================================================
t_train_aligned = t_train[:len(x_train)]
e_train_aligned = e_train[:len(x_train)]

t_test_aligned = t_test[:len(x_test)]
e_test_aligned = e_test[:len(x_test)]
```

```
df_train = pd.DataFrame(x_train[:len(t_train_aligned)])
df_train['t'] = t_train_aligned
df_train['e'] = e_train_aligned

df_test = pd.DataFrame(x_test[:len(t_test_aligned)])
df_test['t'] = t_test_aligned
df_test['e'] = e_test_aligned

cph = CoxPHFitter()
cph.fit(df_train, duration_col='t', event_col='e')
cph_c_index = cph.score(df_test, scoring_method="concordance_index")

# ======================================================================
# 2. Train and Evaluate Random Survival Forest (RSF) Model
# ======================================================================
y_train_struct = np.array(list(zip(e_train_aligned.astype(bool), t_train_aligned)), dtype=[('Status', '?'), ('Survival_in_days', '<f8')])
y_test_struct = np.array(list(zip(e_test_aligned.astype(bool), t_test_aligned)), dtype=[('Status', '?'), ('Survival_in_days', '<f8')])

rsf = RandomSurvivalForest(
    n_estimators=100,
    min_samples_split=10,
    min_samples_leaf=15,
    max_features="sqrt",
    n_jobs=-1,
    random_state=42
)

rsf.fit(x_train[:len(t_train_aligned)], y_train_struct)
rsf_c_index = rsf.score(x_test[:len(t_test_aligned)], y_test_struct)

# ======================================================================
# 3. Print Model Performance Comparison Results
# ======================================================================
print("Model Performance Comparison:")
print(f"1. CPH Test C-index: {cph_c_index:.4f}")
print(f"2. RSF Test C-index: {rsf_c_index:.4f}")
```

```
Model Performance Comparison:
1. CPH Test C-index: 0.6324
2. RSF Test C-index: 0.6443
```

Research Question 1: Does DeepSurv perform effectively on a domain-informed dataset?

B. Test Models - Cirrhosis Dataset

B-1. Load Cirrhosis Data

```
# ================================================================
# Load Cirrhosis Clinical Dataset
# ================================================================
import os
```

```
import pandas as pd

# Define file path (adjust the path if loaded from Google Drive)
file_path = '/content/drive/MyDrive/cirrhosis.csv'

# Check if the file exists and load into DataFrame
if os.path.exists(file_path):
    df_ch = pd.read_csv(file_path)
else:
    # Fallback to local root path
    df_ch = pd.read_csv('cirrhosis.csv')

# Inspect the first 3 patient records
print("First 5 samples of Cirrhosis dataset:")
df_ch.head(5)
```

```
First 5 samples of Cirrhosis dataset:
```

	ID	N_Days	Status	Drug	Age	Sex	Ascites	Hepatomegaly	Spiders	Edema	Bilirubin	Cholesterol	Albumin	Copper	Alk_Phos	SGOT	Tryglicerides	Platelets	Prothrombin	S
0	1	400	D	D-penicillamine	21464	F	Y	Y	Y	Y	14.5	261.0	2.60	156.0	1718.0	137.95	172.0	190.0	12.2	
1	2	4500	C	D-penicillamine	20617	F	N	Y	Y	N	1.1	302.0	4.14	54.0	7394.8	113.52	88.0	221.0	10.6	
2	3	1012	D	D-penicillamine	25594	M	N	N	N	S	1.4	176.0	3.48	210.0	516.0	96.10	55.0	151.0	12.0	

다음 단계: df_ch 변수로 코드 생성 New interactive sheet

B-2. Preprocess Data

```
# One-Hot-Encoding
binary_cols = ["Ascites", "Hepatomegaly", "Spiders"]

# 'Y' -> 1, 'N' -> 0
df_ch_dum = df_ch.copy()

df_ch_dum = pd.get_dummies(
    df_ch_dum, columns=binary_cols, drop_first=True, dtype=float
)

df_ch_dum.head(5)
```

	ID	N_Days	Status	Drug	Age	Sex	Edema	Bilirubin	Cholesterol	Albumin	Copper	Alk_Phos	SGOT	Tryglicerides	Platelets	Prothrombin	Stage	Ascites_Y	Hepatomegaly_Y
0	1	400	D	D-penicillamine	21464	F	Y	14.5	261.0	2.60	156.0	1718.0	137.95	172.0	190.0	12.2	4.0	1.0	1.0
1	2	4500	C	D-penicillamine	20617	F	N	1.1	302.0	4.14	54.0	7394.8	113.52	88.0	221.0	10.6	3.0	0.0	1.0
2	3	1012	D	D-penicillamine	25594	M	S	1.4	176.0	3.48	210.0	516.0	96.10	55.0	151.0	12.0	4.0	0.0	0.0

다음 단계: df_ch_dum 변수로 코드 생성 New interactive sheet

```
import numpy as np
import pandas as pd
from IPython.display import display

# 1-1. Binary Encoding (Y -> 1.0, N -> 0.0)
df_ch_prep = df_ch.copy()
for col in ["Ascites", "Hepatomegaly", "Spiders"]:
    if col in df_ch_prep.columns:
        df_ch_prep[f"{col}_Y"] = df_ch_prep[col].map({"Y": 1.0, "N": 0.0})

# 1-2. Generate Nonlinear Stage-Weighted Risk (X1)
alpha = np.log(2)
stage_weight = np.exp(alpha * (df_ch_prep["Stage"].astype(float) - 1.0))
biomarker_risk = (
    np.log(df_ch_prep["Cholesterol"].astype(float) + 1.0)
    + np.log(df_ch_prep["Tryglicerides"].astype(float) + 1.0)
    - np.log(df_ch_prep["Albumin"].astype(float) + 1e-5)
)
df_ch_prep["Stage_Weighted_Risk"] = stage_weight * biomarker_risk

# 1-3. Target Processing (Status binarization & missing value dropping)
df_ch_prep["Status"] = df_ch_prep["Status"].map({"D": 1.0, "C": 0.0, "CL": 0.0})

ordered_cols = [
    "Ascites_Y",
    "Hepatomegaly_Y",
    "Spiders_Y",
    "Bilirubin",
    "Copper",
    "Prothrombin",
    "Alk_Phos",
    "SGOT",
    "Platelets",
    "Stage_Weighted_Risk",
    "N_Days",
    "Status"
]

# Drop rows with missing values in the required columns
df_ch_clean = df_ch_prep[ordered_cols].dropna().reset_index(drop=True)

# 1-4. Rename to Table 4 exact headers
rename_headers = {
    "Ascites_Y": "Ascites (Y)",
    "Hepatomegaly_Y": "Hepato (Y)",
    "Spiders_Y": "Spiders (Y)",
    "Alk_Phos": "Alk Phos",
    "Stage_Weighted_Risk": "Stage-Weighted Risk (X1)",
    "N_Days": "Duration (T)",
    "Status": "Event ()"
}

df_ch_table4_single = df_ch_clean.rename(columns=rename_headers).copy()
```

```
df_ch_table4_single["Event ()"] = df_ch_table4_single["Event ()"].astype(int)

df_ch_table4_single["Stage-Weighted Risk (X1)"] = df_ch_table4_single["Stage-Weighted Risk (X1)"].round(2)
df_ch_table4_single["Duration (T)"] = df_ch_table4_single["Duration (T)"].astype(int)

print("Sample Instances of the Domain-Engineered Cirrhosis Patient Survival Dataset")
display(df_ch_table4_single.head(5))
```

Sample Instances of the Domain-Engineered Cirrhosis Patient Survival Dataset

	Ascites (Y)	Hepato (Y)	Spiders (Y)	Bilirubin	Copper	Prothrombin	Alk Phos	SGOT	Platelets	Stage-Weighted Risk (X1)	Duration (T)	Event ()
0	1.0	1.0	1.0	14.5	156.0	12.2	1718.0	137.95	190.0	78.13	400	1
1	0.0	1.0	1.0	1.1	54.0	10.6	7394.8	113.52	221.0	35.13	4500	0
2	0.0	0.0	0.0	1.4	210.0	12.0	516.0	96.10	151.0	63.64	1012	1
3	0.0	1.0	1.0	1.8	64.0	10.3	6121.8	60.63	183.0	72.81	1925	1
4	0.0	1.0	1.0	3.4	143.0	10.9	671.0	113.15	136.0	34.66	1504	0

B-3. Test Models on the domain-guided cirrhosis survival dataset

```
# Set random seed for reproducibility
SEED = 48764388
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# ================================================================
# 1. Feature Engineering and Multivariate Preprocessing
# ================================================================
df_ch_wt = df_ch_dum.copy()

# 1-1. Generate nonlinear Stage-Weighted Risk feature
alpha = np.log(2)
stage_weight = np.exp(alpha * (df_ch_wt["Stage"] - 1.0))
biomarker_risk = (
    np.log(df_ch_wt["Cholesterol"] + 1.0)
    + np.log(df_ch_wt["Tryglicerides"] + 1.0)
    - np.log(df_ch_wt["Albumin"] + 1e-5)
)
df_ch_wt["Stage_Weighted_Risk"] = stage_weight * biomarker_risk

# 1-2. Define final 10-feature set
columns_to_keep = [
    "N_Days",
    "Status",
    "Ascites_Y",
    "Hepatomegaly_Y",
    "Spiders_Y",
    "Stage_Weighted_Risk",
    "Bilirubin",
    "Copper",
    "Prothrombin",
```

```
    "Alk_Phos",
    "SGOT",
    "Platelets"
]

df_ch_final = df_ch_wt[[c for c in columns_to_keep if c in df_ch_wt.columns]].copy()

# 1-3. Binarize Status (D: 1.0, C/CL: 0.0) and handle missing values
df_ch_final["Status"] = df_ch_final["Status"].map({"D": 1.0, "C": 0.0, "CL": 0.0})
df_ch_final = df_ch_final.dropna().reset_index(drop=True)

df_ch_final["Status"] = df_ch_final["Status"].astype("float32")
df_ch_final["N_Days"] = df_ch_final["N_Days"].astype("float32")

# ================================================================
# 2. Stratified Data Splitting and Feature Scaling
# ================================================================
feature_cols = [col for col in df_ch_final.columns if col not in ["N_Days", "Status"]]
X = df_ch_final[feature_cols].values.astype("float32")
durations = df_ch_final["N_Days"].values.astype("float32")
events = df_ch_final["Status"].values.astype("float32")

# Step 1: Split into Train+Val (85%) vs Test (15%)
X_train_full, X_test, d_train_full, d_test, e_train_full, e_test = train_test_split(
    X, durations, events, test_size=0.15, random_state=SEED, stratify=events
)

# Step 2: Split Train+Val into Train (70%) vs Val (15%) - For DeepSurv Early Stopping
X_train, X_val, d_train, d_val, e_train, e_val = train_test_split(
    X_train_full, d_train_full, e_train_full, test_size=0.17647, random_state=SEED, stratify=e_train_full
)

# Standardize features (Fit on Train only to prevent data leakage)
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_train_scaled = scaler.transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# ================================================================
# 3. [Model 1] Cox Proportional Hazards (CPH)
# ================================================================
df_train_cph = pd.DataFrame(X_train_full_scaled, columns=feature_cols)
df_train_cph["t"] = d_train_full
df_train_cph["e"] = e_train_full

df_test_cph = pd.DataFrame(X_test_scaled, columns=feature_cols)
df_test_cph["t"] = d_test
df_test_cph["e"] = e_test

cph = CoxPHFitter(penalizer=0.05)
cph.fit(df_train_cph, duration_col="t", event_col="e")

# Predict partial hazard log-ratio and compute Concordance Index
cph_train_risk = cph.predict_log_partial_hazard(df_train_cph).values.flatten()
cph_test_risk = cph.predict_log_partial_hazard(df_test_cph).values.flatten()
```

```
cph_train_c = concordance_index_censored(e_train_full.astype(bool), d_train_full, cph_train_risk)[0]
cph_test_c = concordance_index_censored(e_test.astype(bool), d_test, cph_test_risk)[0]

# ================================================================
# 4. [Model 2] Random Survival Forest (RSF)
# ================================================================
y_train_struct = np.array(
    list(zip(e_train_full.astype(bool), d_train_full)),
    dtype=[("Status", "?"), ("Survival_in_days", "<f8")]
)
y_test_struct = np.array(
    list(zip(e_test.astype(bool), d_test)),
    dtype=[("Status", "?"), ("Survival_in_days", "<f8")]
)

rsf = RandomSurvivalForest(
    n_estimators=100,
    min_samples_split=10,
    min_samples_leaf=15,
    max_features="sqrt",
    n_jobs=-1,
    random_state=SEED
)
rsf.fit(X_train_full_scaled, y_train_struct)

rsf_train_c = rsf.score(X_train_full_scaled, y_train_struct)
rsf_test_c = rsf.score(X_test_scaled, y_test_struct)

# ================================================================
# 5. [Model 3] DeepSurv (Deep Neural Network)
# ================================================================
y_train_deep = (d_train, e_train)
y_val_deep = (d_val, e_val)
val_data = (X_val_scaled, y_val_deep)

in_features = X_train_scaled.shape[1]
net = tt.practical.MLPVanilla(
    in_features,
    [32, 16],
    out_features=1,
    batch_norm=True,
    dropout=0.2,
    activation=nn.SELU
)

model_deep = CoxPH(net, tt.optim.Adam(lr=0.002, weight_decay=1e-3))
callbacks = [tt.callbacks.EarlyStopping(patience=20)]

log = model_deep.fit(
    X_train_scaled, y_train_deep,
    batch_size=64,
    epochs=200,
    callbacks=callbacks,
    val_data=val_data,
    verbose=False
```

```
)

# Predict risk scores and calibrate risk score direction
risk_train_deep = model_deep.predict(X_train_scaled).flatten()
risk_test_deep = model_deep.predict(X_test_scaled).flatten()

if risk_test_deep[e_test == 1].mean() < risk_test_deep[e_test == 0].mean():
    risk_train_deep = -risk_train_deep
    risk_test_deep = -risk_test_deep

deepsurv_train_c = concordance_index_censored(e_train.astype(bool), d_train, risk_train_deep)[0]
deepsurv_test_c = concordance_index_censored(e_test.astype(bool), d_test, risk_test_deep)[0]

# ======================================================================
# 6. Final Performance Benchmark Table
# ======================================================================
summary_data = {
    "Model": ["Cox Proportional Hazards (CPH)", "Random Survival Forest (RSF)", "DeepSurv (Neural Network)"],
    "Model Family": ["Semiparametric / Linear", "Nonlinear Tree Ensemble", "Deep Neural Network"],
    "Train C-Index": [f"{cph_train_c:.4f}", f"{rsf_train_c:.4f}", f"{deepsurv_train_c:.4f}"],
    "Test C-Index": [f"{cph_test_c:.4f}", f"{rsf_test_c:.4f}", f"{deepsurv_test_c:.4f}"]
}

df_results = pd.DataFrame(summary_data)

print(f"[Cirrhosis Dataset] Survival Benchmark Comparison")
print(df_results.to_string(index=False))
```

```
[Cirrhosis Dataset] Survival Benchmark Comparison
                        Model            Model Family Train C-Index Test C-Index
Cox Proportional Hazards (CPH) Semiparametric / Linear        0.8481       0.7393
  Random Survival Forest (RSF) Nonlinear Tree Ensemble        0.8657       0.7180
     DeepSurv (Neural Network)     Deep Neural Network        0.8583       0.7370
```

C. Test Models - German Credit Dataset

C-1. Load German Bank Data

```
# ======================================================================
# Load German Credit Dataset (german.data)
# ======================================================================
import os
import pandas as pd

# Set Pandas options to display all columns without truncation
pd.set_option('display.max_columns', None)       # Display all columns
pd.set_option('display.width', 1000)             # Increase display width to prevent line wrapping
pd.set_option('display.max_colwidth', None)      # Do not truncate content inside cells

# Define path (adjust if stored inside Google Drive)
file_path_german = '/content/drive/MyDrive/german.data'

if not os.path.exists(file_path_german):
    file_path_german = 'german.data'
```

```
# Standard semantic column names according to UCI attribute specification
german_cols = [
    "Status_Checking_Acc",
    "Duration_in_month",
    "Credit_History",
    "Purpose",
    "Credit_Amount",
    "Savings_Account",
    "Present_Employment_Since",
    "Installment_Rate",
    "Personal_Status_Sex",
    "Other_Debtors",
    "Present_Residence_Since",
    "Property",
    "Age_in_years",
    "Other_Installment_Plans",
    "Housing",
    "Existing_Credits",
    "Job",
    "Num_People_Liable",
    "Telephone",
    "Foreign_Worker",
    "Credit_Risk"  # Target (1 = Good, 2 = Bad)
]

# Load using whitespace delimiter
df_german = pd.read_csv(file_path_german, sep=r'\s+', header=None, names=german_cols)

print(f"Successfully loaded German Credit Data (Rows: {df_german.shape[0]}, Columns: {df_german.shape[1]}):")
display(df_german.head(5))
```

```
Successfully loaded German Credit Data (Rows: 1000, Columns: 21):
```

	Status_Checking_Acc	Duration_in_month	Credit_History	Purpose	Credit_Amount	Savings_Account	Present_Employment_Since	Installment_Rate	Personal_Status_Sex	Other_Debtors	Pre
0	A11	6	A34	A43	1169	A65	A75	4	A93	A101	
1	A12	48	A32	A43	5951	A61	A73	2	A92	A101	
2	A14	12	A34	A46	2096	A61	A74	2	A93	A101	
3	A11	42	A32	A42	7882	A61	A74	2	A93	A103	
4	A11	24	A33	A40	4870	A61	A73	3	A93	A101	

C-2. Preprocess Data

```
# ================================================================
# Load and Encode German Credit Dataset (german.data)
# ================================================================
import os
import pandas as pd

# Set Pandas options to display all columns and prevent truncation
pd.set_option('display.max_columns', None)       # Display all columns
pd.set_option('display.width', 1000)             # Increase display width to prevent line wrapping
```

```
pd.set_option('display.max_colwidth', None)      # Do not truncate content inside cells

# Define path (adjust if stored inside Google Drive)
file_path_german = '/content/drive/MyDrive/german.data'

if not os.path.exists(file_path_german):
    file_path_german = 'german.data'

# Standard semantic column names according to UCI attribute specification
german_cols = [
    "Status_Checking_Acc",
    "Duration_in_month",
    "Credit_History",
    "Purpose",
    "Credit_Amount",
    "Savings_Account",
    "Present_Employment_Since",
    "Installment_Rate",
    "Personal_Status_Sex",
    "Other_Debtors",
    "Present_Residence_Since",
    "Property",
    "Age_in_years",
    "Other_Installment_Plans",
    "Housing",
    "Existing_Credits",
    "Job",
    "Num_People_Liable",
    "Telephone",
    "Foreign_Worker",
    "Credit_Risk"  # Target (1 = Good, 2 = Bad)
]

# Load using whitespace delimiter
df_german = pd.read_csv(file_path_german, sep=r'\s+', header=None, names=german_cols)

# Copy dataframe for encoding
df_encoded = df_german.copy()

# Define mapping dictionaries for categorical features
cs_map = {"A11": 1.0, "A12": 2.0, "A13": 3.0, "A14": 4.0}
ss_map = {"A61": 1.0, "A62": 2.0, "A63": 3.0, "A64": 4.0, "A65": 1.0}
es_map = {"A71": 0.0, "A72": 1.0, "A73": 2.0, "A74": 3.0, "A75": 4.0}
ps_map = {"A121": 4.0, "A122": 3.0, "A123": 2.0, "A124": 1.0}
gs_map = {"A101": 1.0, "A102": 2.0, "A103": 3.0}

# Apply mappings to encoded dataframe
df_encoded["Status_Checking_Acc"] = df_encoded["Status_Checking_Acc"].map(cs_map).fillna(1.0)
df_encoded["Savings_Account"] = df_encoded["Savings_Account"].map(ss_map).fillna(1.0)
df_encoded["Present_Employment_Since"] = df_encoded["Present_Employment_Since"].map(es_map).fillna(0.0)
df_encoded["Property"] = df_encoded["Property"].map(ps_map).fillna(1.0)
df_encoded["Other_Debtors"] = df_encoded["Other_Debtors"].map(gs_map).fillna(1.0)

print(f"Successfully loaded and mapped German Credit Data (Rows: {df_encoded.shape[0]}, Columns: {df_encoded.shape[1]}:")
display(df_encoded.head(5))
```

Successfully loaded and mapped German Credit Data (Rows: 1000, Columns: 21:

	Status_Checking_Acc	Duration_in_month	Credit_History	Purpose	Credit_Amount	Savings_Account	Present_Employment_Since	Installment_Rate	Personal_Status_Sex	Other_Debtors	Pre
0	1.0	6	A34	A43	1169	1.0	4.0	4	A93	1.0	
1	2.0	48	A32	A43	5951	1.0	2.0	2	A92	1.0	
2	4.0	12	A34	A46	2096	1.0	3.0	2	A93	1.0	
3	1.0	42	A32	A42	7882	1.0	3.0	2	A93	3.0	
4	1.0	24	A33	A40	4870	1.0	2.0	3	A93	1.0	

```
# ================================================================
# Domain Feature Engineering Pipeline (Final Unified Version)
# ================================================================
import numpy as np
import pandas as pd
from IPython.display import display

df = df_german.copy()

# 1. Define Survival Analysis Target Variables (Time, Event)
# Event indicator: 1 = Good (0, Censored), 2 = Bad (1, Default Event)
df["Duration (Months) (T)"] = df["Duration_in_month"].astype(float)
df["Event (δ)"] = (df["Credit_Risk"] == 2).astype(int)

# 2. Domain Categorical Ordinal Score Mappings
cs_map = {"A11": 1.0, "A12": 2.0, "A13": 3.0, "A14": 4.0}
ss_map = {"A61": 1.0, "A62": 2.0, "A63": 3.0, "A64": 4.0, "A65": 1.0}
es_map = {"A71": 0.0, "A72": 1.0, "A73": 2.0, "A74": 3.0, "A75": 4.0}
ps_map = {"A121": 4.0, "A122": 3.0, "A123": 2.0, "A124": 1.0}
gs_map = {"A101": 1.0, "A102": 2.0, "A103": 3.0}

df["CS"] = df["Status_Checking_Acc"].map(cs_map).fillna(1.0)
df["SS"] = df["Savings_Account"].map(ss_map).fillna(1.0)
df["ES"] = df["Present_Employment_Since"].map(es_map).fillna(0.0)
df["PS"] = df["Property"].map(ps_map).fillna(1.0)
df["GS"] = df["Other_Debtors"].map(gs_map).fillna(1.0)

epsilon = 1e-5

# 3. Generate 4 Domain Features Based on Paper Formulations (Eq. 3 - Eq. 6)
# (1) Financial Burden Index (X1) - Eq. 3
df["Financial Burden Index (X1)"] = df["Installment_Rate"] * np.log(
    (df["Credit_Amount"] / (df["CS"] * df["SS"] + epsilon)) + 1.0
)

# (2) Asset and Employment Stability (X2) - Eq. 4
df["Asset and Employment Stability (X2)"] = df["PS"] * df["ES"]

# (3) Net Capacity per Dependent (X3) - Eq. 5
df["Net Capacity per Dependent (X3)"] = (
    df["Age_in_years"].astype(float) + df["Present_Residence_Since"].astype(float)
) / df["Num_People_Liable"].astype(float)
```

```
# (4) Guarantor-Adjusted Credit (X4) - Eq. 6
df["Guarantor-Adjusted Credit (X4)"] = (
    df["Credit_Amount"].astype(float) / (df["GS"] + epsilon)
)

# 4. Construct Final Feature Matrix for Survival Analysis
final_cols = [
    "Financial Burden Index (X1)",
    "Asset and Employment Stability (X2)",
    "Net Capacity per Dependent (X3)",
    "Guarantor-Adjusted Credit (X4)",
    "Duration (Months) (T)",
    "Event (δ)"
]

df_domain_final = df[final_cols].copy()

df_domain_final["Financial Burden Index (X1)"] = df_domain_final["Financial Burden Index (X1)"].round(2)
df_domain_final["Guarantor-Adjusted Credit (X4)"] = df_domain_final["Guarantor-Adjusted Credit (X4)"].round(2)
df_domain_final["Duration (Months) (T)"] = df_domain_final["Duration (Months) (T)"].astype(int)

display(df_domain_final.head(5))
```

	Financial Burden Index (X1)	Asset and Employment Stability (X2)	Net Capacity per Dependent (X3)	Guarantor-Adjusted Credit (X4)	Duration (Months) (T)	Event (δ)
0	28.26	16.0	71.0	1168.99	6	0
1	16.00	8.0	24.0	5950.94	48	1
2	12.53	12.0	26.0	2095.98	12	0
3	17.94	9.0	24.5	2627.32	42	0
4	25.47	2.0	28.5	4869.95	24	1

C-3. Test Models on the domain-guided German credit dataset

```
# ==========================================================================
# 0. Set Random Seed for Reproducibility
# ==========================================================================
SEED = 48764388
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# ==========================================================================
# 1. Domain Feature Engineering Pipeline
# ==========================================================================
df = df_german.copy()

# 1-1. Define Target Variables (T: Duration in Months, Event: 1 for Bad, 0 for Good)
df["Duration (Months) (T)"] = df["Duration_in_month"].astype("float32")
df["Event (δ)"] = (df["Credit_Risk"] == 2).astype("float32")
```

```
# 1-2. Domain Categorical Ordinal Score Mappings (Paper Specification)
cs_map = {"A11": 1.0, "A12": 2.0, "A13": 3.0, "A14": 4.0}
ss_map = {"A61": 1.0, "A62": 2.0, "A63": 3.0, "A64": 4.0, "A65": 1.0}
es_map = {"A71": 0.0, "A72": 1.0, "A73": 2.0, "A74": 3.0, "A75": 4.0}
ps_map = {"A121": 4.0, "A122": 3.0, "A123": 2.0, "A124": 1.0}
gs_map = {"A101": 1.0, "A102": 2.0, "A103": 3.0}

df["CS"] = df["Status_Checking_Acc"].map(cs_map).fillna(1.0)
df["SS"] = df["Savings_Account"].map(ss_map).fillna(1.0)
df["ES"] = df["Present_Employment_Since"].map(es_map).fillna(0.0)
df["PS"] = df["Property"].map(ps_map).fillna(1.0)
df["GS"] = df["Other_Debtors"].map(gs_map).fillna(1.0)

epsilon = 1e-5

# 1-3. Generate 4 Domain Features
# Eq. 3: Financial Burden Index (X1)
df["Financial Burden Index (X1)"] = df["Installment_Rate"] * np.log(
    (df["Credit_Amount"] / (df["CS"] * df["SS"] + epsilon)) + 1.0
)

# Eq. 4: Asset and Employment Stability (X2)
df["Asset and Employment Stability (X2)"] = df["PS"] * df["ES"]

# Eq. 5: Net Capacity per Dependent (X3)
df["Net Capacity per Dependent (X3)"] = (
    df["Age_in_years"].astype(float) + df["Present_Residence_Since"].astype(float)
) / df["Num_People_Liable"].astype(float)

# Eq. 6: Guarantor-Adjusted Credit (X4)
df["Guarantor-Adjusted Credit (X4)"] = (
    df["Credit_Amount"].astype(float) / (df["GS"] + epsilon)
)

# 1-4. Construct Final Feature Matrix
feature_cols = [
    "Financial Burden Index (X1)",
    "Asset and Employment Stability (X2)",
    "Net Capacity per Dependent (X3)",
    "Guarantor-Adjusted Credit (X4)"
]
final_cols = feature_cols + ["Duration (Months) (T)", "Event ()"]

df_domain_final = df[final_cols].dropna().reset_index(drop=True)
# display(df_domain_final.head(5))

# ================================================================
# 2. Stratified Data Splitting & Feature Standardization
# ================================================================
X = df_domain_final[feature_cols].values.astype("float32")
durations = df_domain_final["Duration (Months) (T)"].values.astype("float32")
events = df_domain_final["Event ()"].values.astype("float32")

# Step 1: Split into Train+Val (85%) vs Test (15%)
X_train_full, X_test, d_train_full, d_test, e_train_full, e_test = train_test_split(
    X, durations, events, test_size=0.15, random_state=SEED, stratify=events
```

```
)

# Step 2: Split Train+Val into Train (70%) vs Val (15%) for DeepSurv Early Stopping
X_train, X_val, d_train, d_val, e_train, e_val = train_test_split(
    X_train_full, d_train_full, e_train_full, test_size=0.17647, random_state=SEED, stratify=e_train_full
)

# Feature standardization (fitted strictly on training set to prevent leakage)
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_train_scaled = scaler.transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# ======================================================================
# 3. [Model 1] Cox Proportional Hazards (CPH)
# ======================================================================
df_train_cph = pd.DataFrame(X_train_full_scaled, columns=feature_cols)
df_train_cph["t"] = d_train_full
df_train_cph["e"] = e_train_full

df_test_cph = pd.DataFrame(X_test_scaled, columns=feature_cols)
df_test_cph["t"] = d_test
df_test_cph["e"] = e_test

cph = CoxPHFitter(penalizer=0.05)
cph.fit(df_train_cph, duration_col="t", event_col="e")

cph_train_risk = cph.predict_log_partial_hazard(df_train_cph).values.flatten()
cph_test_risk = cph.predict_log_partial_hazard(df_test_cph).values.flatten()

cph_train_c = concordance_index_censored(e_train_full.astype(bool), d_train_full, cph_train_risk)[0]
cph_test_c = concordance_index_censored(e_test.astype(bool), d_test, cph_test_risk)[0]

# ======================================================================
# 4. [Model 2] Random Survival Forest (RSF)
# ======================================================================
y_train_struct = np.array(
    list(zip(e_train_full.astype(bool), d_train_full)),
    dtype=[("Status", "?"), ("Survival_in_days", "<f8")]
)
y_test_struct = np.array(
    list(zip(e_test.astype(bool), d_test)),
    dtype=[("Status", "?"), ("Survival_in_days", "<f8")]
)

rsf = RandomSurvivalForest(
    n_estimators=100,
    min_samples_split=10,
    min_samples_leaf=15,
    max_features="sqrt",
    n_jobs=-1,
    random_state=SEED
)
rsf.fit(X_train_full_scaled, y_train_struct)
```

```
rsf_train_c = rsf.score(X_train_full_scaled, y_train_struct)
rsf_test_c = rsf.score(X_test_scaled, y_test_struct)

# ==========================================================
# 5. [Model 3] DeepSurv (Neural Network) - Calibrated Risk Score
# ==========================================================
y_train_deep = (d_train, e_train)
y_val_deep = (d_val, e_val)
val_data = (X_val_scaled, y_val_deep)

in_features = X_train_scaled.shape[1]
net = tt.practical.MLPVanilla(
    in_features,
    [32, 16],
    out_features=1,
    batch_norm=True,
    dropout=0.2,
    activation=nn.SELU
)

model_deep = CoxPH(net, tt.optim.Adam(lr=0.002, weight_decay=1e-3))
callbacks = [tt.callbacks.EarlyStopping(patience=20)]

log = model_deep.fit(
    X_train_scaled, y_train_deep,
    batch_size=64,
    epochs=200,
    callbacks=callbacks,
    val_data=val_data,
    verbose=False
)

# pycox outputs log-hazard (higher score = higher hazard)
risk_train_deep = model_deep.predict(X_train_scaled).flatten()
risk_test_deep = model_deep.predict(X_test_scaled).flatten()

deepsurv_train_c = concordance_index_censored(e_train.astype(bool), d_train, risk_train_deep)[0]
deepsurv_test_c = concordance_index_censored(e_test.astype(bool), d_test, risk_test_deep)[0]

# ==========================================================
# 6. Benchmark Performance Table
# ==========================================================
summary_data = {
    "Model": ["Cox Proportional Hazards (CPH)", "Random Survival Forest (RSF)", "DeepSurv (Neural Network)"],
    "Model Family": ["Semiparametric / Linear", "Nonlinear Tree Ensemble", "Deep Neural Network"],
    "Train C-Index": [f"{cph_train_c:.4f}", f"{rsf_train_c:.4f}", f"{deepsurv_train_c:.4f}"],
    "Test C-Index": [f"{cph_test_c:.4f}", f"{rsf_test_c:.4f}", f"{deepsurv_test_c:.4f}"]
}

df_results = pd.DataFrame(summary_data)

print(f"[German Credit Dataset] Survival Benchmark Comparison")
print(df_results.to_string(index=False))
```

```
[German Credit Dataset] Survival Benchmark Comparison
                        Model            Model Family Train C-Index Test C-Index
Cox Proportional Hazards (CPH) Semiparametric / Linear        0.7027       0.6388
```

```
Random Survival Forest (RSF) Nonlinear Tree Ensemble        0.8159       0.6906
   DeepSurv (Neural Network)     Deep Neural Network        0.7255       0.7021
```

Research Question 2: Does DeepSurv outperform its performance on a raw dataset when using a domain-informed dataset?

D. Test Models - Cirrhosis Dataset

```
# ================================================================
# 0. Set random seed for reproducibility
# ================================================================
SEED = 48764388
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# ================================================================
# 1. Strictly Selected Raw Features (Extracting 13 raw features used in engineering)
# ================================================================
exact_raw_feature_cols = [
    "Ascites_Y",
    "Hepatomegaly_Y",
    "Spiders_Y",
    "Bilirubin",
    "Copper",
    "Prothrombin",
    "Alk_Phos",
    "SGOT",
    "Platelets",
    "Stage",
    "Cholesterol",
    "Tryglicerides",
    "Albumin"
]

target_cols = ["N_Days", "Status"]
columns_to_keep = target_cols + exact_raw_feature_cols

df_ch_raw_exact = df_ch_dum[[c for c in columns_to_keep if c in df_ch_dum.columns]].copy()

# Target binarization (D: 1.0, C/CL: 0.0)
df_ch_raw_exact["Status"] = df_ch_raw_exact["Status"].map({"D": 1.0, "C": 0.0, "CL": 0.0})

# Drop missing values and reset index to completely prevent NaN issues
df_ch_raw_exact = df_ch_raw_exact.dropna().reset_index(drop=True)

df_ch_raw_exact["Status"] = df_ch_raw_exact["Status"].astype("float32")
df_ch_raw_exact["N_Days"] = df_ch_raw_exact["N_Days"].astype("float32")

X = df_ch_raw_exact[exact_raw_feature_cols].values.astype("float32")
durations = df_ch_raw_exact["N_Days"].values.astype("float32")
events = df_ch_raw_exact["Status"].values.astype("float32")
```

```
# ================================================================
# 2. Stratified Data Splitting & Feature Scaling (Identical splitting)
# ================================================================
# Step 1: Split into Train+Val (85%) vs Test (15%)
X_train_full, X_test, d_train_full, d_test, e_train_full, e_test = train_test_split(
    X, durations, events, test_size=0.15, random_state=SEED, stratify=events
)

# Step 2: Split Train+Val into Train (70%) vs Val (15%)
X_train, X_val, d_train, d_val, e_train, e_val = train_test_split(
    X_train_full, d_train_full, e_train_full, test_size=0.17647, random_state=SEED, stratify=e_train_full
)

# Standardize features (fit based on training data)
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_train_scaled = scaler.transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# ================================================================
# 3. [Model 1] Cox Proportional Hazards (CPH)
# ================================================================
df_train_cph = pd.DataFrame(X_train_full_scaled, columns=exact_raw_feature_cols)
df_train_cph['t'] = d_train_full
df_train_cph['e'] = e_train_full

df_test_cph = pd.DataFrame(X_test_scaled, columns=exact_raw_feature_cols)
df_test_cph['t'] = d_test
df_test_cph['e'] = e_test

cph = CoxPHFitter(penalizer=0.05)
cph.fit(df_train_cph, duration_col='t', event_col='e')

cph_train_risk = cph.predict_log_partial_hazard(df_train_cph).values.flatten()
cph_test_risk = cph.predict_log_partial_hazard(df_test_cph).values.flatten()

cph_train_c = concordance_index_censored(e_train_full.astype(bool), d_train_full, cph_train_risk)[0]
cph_test_c = concordance_index_censored(e_test.astype(bool), d_test, cph_test_risk)[0]

# ================================================================
# 4. [Model 2] Random Survival Forest (RSF)
# ================================================================
y_train_struct = np.array(
    list(zip(e_train_full.astype(bool), d_train_full)),
    dtype=[('Status', '?'), ('Survival_in_days', '<f8')]
)
y_test_struct = np.array(
    list(zip(e_test.astype(bool), d_test)),
    dtype=[('Status', '?'), ('Survival_in_days', '<f8')]
)

rsf = RandomSurvivalForest(
    n_estimators=100,
    min_samples_split=10,
```

```
    min_samples_leaf=15,
    max_features="sqrt",
    n_jobs=-1,
    random_state=SEED
)
rsf.fit(X_train_full_scaled, y_train_struct)

rsf_train_c = rsf.score(X_train_full_scaled, y_train_struct)
rsf_test_c = rsf.score(X_test_scaled, y_test_struct)

# ==========================================================================
# 5. [Model 3] DeepSurv (Deep Neural Network)
# ==========================================================================
y_train_deep = (d_train, e_train)
y_val_deep = (d_val, e_val)
val_data = (X_val_scaled, y_val_deep)

in_features = X_train_scaled.shape[1]
net = tt.practical.MLPVanilla(
    in_features,
    [32, 16],
    out_features=1,
    batch_norm=True,
    dropout=0.2,
    activation=nn.SELU
)

model_deep = CoxPH(net, tt.optim.Adam(lr=0.002, weight_decay=1e-3))
callbacks = [tt.callbacks.EarlyStopping(patience=20)]

log = model_deep.fit(
    X_train_scaled, y_train_deep,
    batch_size=64,
    epochs=200,
    callbacks=callbacks,
    val_data=val_data,
    verbose=False
)

risk_train_deep = model_deep.predict(X_train_scaled).flatten()
risk_test_deep = model_deep.predict(X_test_scaled).flatten()

if risk_test_deep[e_test == 1].mean() < risk_test_deep[e_test == 0].mean():
    risk_train_deep = -risk_train_deep
    risk_test_deep = -risk_test_deep

deepsurv_train_c = concordance_index_censored(e_train.astype(bool), d_train, risk_train_deep)[0]
deepsurv_test_c = concordance_index_censored(e_test.astype(bool), d_test, risk_test_deep)[0]

# ==========================================================================
# 6. Final Performance Benchmark Table (Pure Raw Decomposition)
# ==========================================================================
summary_data = {
    "Model": ["Cox Proportional Hazards (CPH)", "Random Survival Forest (RSF)", "DeepSurv (Neural Network)"],
    "Model Family": ["Semiparametric / Linear", "Nonlinear Tree Ensemble", "Deep Neural Network"],
    "Train C-Index": [f"{cph_train_c:.4f}", f"{rsf_train_c:.4f}", f"{deepsurv_train_c:.4f}"],
```

```
    "Test C-Index": [f"{cph_test_c:.4f}", f"{rsf_test_c:.4f}", f"{deepsurv_test_c:.4f}"]
}

df_results_pure_raw = pd.DataFrame(summary_data)


print(f"[Cirrhosis Exact Raw Decomposition] Benchmark")
print(df_results_pure_raw.to_string(index=False))
```

```
[Cirrhosis Exact Raw Decomposition] Benchmark
                      Model            Model Family Train C-Index Test C-Index
Cox Proportional Hazards (CPH) Semiparametric / Linear        0.8458       0.7654
  Random Survival Forest (RSF) Nonlinear Tree Ensemble        0.8715       0.7251
     DeepSurv (Neural Network)     Deep Neural Network        0.8609       0.7678
```

E. Test Models - German Credit Dataset

```
# ====================================================================
# 0. Set random seed for reproducibility
# ====================================================================
SEED = 48764388
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# ====================================================================
# 1. Fair Raw Decomposition (Extracting only 10 raw variables directly fed into formula calculations)
# ====================================================================
df_fair_german = df_german.copy()

# 1-1. Target Variables
df_fair_german["Duration (Months) (T)"] = df_fair_german["Duration_in_month"].astype("float32")
df_fair_german["Event (δ)"] = (df_fair_german["Credit_Risk"] == 2).astype("float32")

# 1-2. Categorical ordinal scores for paper formula mapping (Preserving ordinal scale conversion)
cs_map = {"A11": 1.0, "A12": 2.0, "A13": 3.0, "A14": 4.0}
ss_map = {"A61": 1.0, "A62": 2.0, "A63": 3.0, "A64": 4.0, "A65": 1.0}
es_map = {"A71": 0.0, "A72": 1.0, "A73": 2.0, "A74": 3.0, "A75": 4.0}
ps_map = {"A121": 4.0, "A122": 3.0, "A123": 2.0, "A124": 1.0}
gs_map = {"A101": 1.0, "A102": 2.0, "A103": 3.0}

df_fair_german["CS"] = df_fair_german["Status_Checking_Acc"].map(cs_map).fillna(1.0)
df_fair_german["SS"] = df_fair_german["Savings_Account"].map(ss_map).fillna(1.0)
df_fair_german["ES"] = df_fair_german["Present_Employment_Since"].map(es_map).fillna(0.0)
df_fair_german["PS"] = df_fair_german["Property"].map(ps_map).fillna(1.0)
df_fair_german["GS"] = df_fair_german["Other_Debtors"].map(gs_map).fillna(1.0)

# 1-3. List of 10 pure raw variables used in calculations (excluding unrelated variables)
raw_decomposed_cols = [
    # Numerical raw variables (5)
    "Credit_Amount",
    "Installment_Rate",
    "Age_in_years",
    "Present_Residence_Since",
    "Num_People_Liable",
```

```
    # Ordinal raw scores used in formulas (5)
    "CS",
    "SS",
    "ES",
    "PS",
    "GS"
]

target_cols = ["Duration (Months) (T)", "Event (δ)"]
all_cols = target_cols + raw_decomposed_cols

df_fair_german = df_fair_german[all_cols].dropna().reset_index(drop=True)

for col in raw_decomposed_cols:
    df_fair_german[col] = df_fair_german[col].astype("float32")

# ==========================================================================
# 2. Stratified Data Splitting & Feature Scaling (Identical splitting)
# ==========================================================================
feature_cols = raw_decomposed_cols
X = df_fair_german[feature_cols].values.astype("float32")
durations = df_fair_german["Duration (Months) (T)"].values.astype("float32")
events = df_fair_german["Event (δ)"].values.astype("float32")

# Train+Val (85%) vs Test (15%)
X_train_full, X_test, d_train_full, d_test, e_train_full, e_test = train_test_split(
    X, durations, events, test_size=0.15, random_state=SEED, stratify=events
)

# Train (70%) vs Val (15%)
X_train, X_val, d_train, d_val, e_train, e_val = train_test_split(
    X_train_full, d_train_full, e_train_full, test_size=0.17647, random_state=SEED, stratify=e_train_full
)

# Standardize features (fit based on training data)
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_train_scaled = scaler.transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# ==========================================================================
# 3. [Model 1] Cox Proportional Hazards (CPH)
# ==========================================================================
df_train_cph = pd.DataFrame(X_train_full_scaled, columns=feature_cols)
df_train_cph['t'] = d_train_full
df_train_cph['e'] = e_train_full

df_test_cph = pd.DataFrame(X_test_scaled, columns=feature_cols)
df_test_cph['t'] = d_test
df_test_cph['e'] = e_test

cph = CoxPHFitter(penalizer=0.05)
cph.fit(df_train_cph, duration_col='t', event_col='e')

cph_train_risk = cph.predict_log_partial_hazard(df_train_cph).values.flatten()
cph_test_risk = cph.predict_log_partial_hazard(df_test_cph).values.flatten()
```

```
cph_test_risk = cph.predict_log_partial_hazard(df_test_cph).values.flatten()

cph_train_c = concordance_index_censored(e_train_full.astype(bool), d_train_full, cph_train_risk)[0]
cph_test_c = concordance_index_censored(e_test.astype(bool), d_test, cph_test_risk)[0]

# ================================================================
# 4. [Model 2] Random Survival Forest (RSF)
# ================================================================
y_train_struct = np.array(
    list(zip(e_train_full.astype(bool), d_train_full)),
    dtype=[('Status', '?'), ('Survival_in_days', '<f8')]
)
y_test_struct = np.array(
    list(zip(e_test.astype(bool), d_test)),
    dtype=[('Status', '?'), ('Survival_in_days', '<f8')]
)

rsf = RandomSurvivalForest(
    n_estimators=100,
    min_samples_split=10,
    min_samples_leaf=15,
    max_features="sqrt",
    n_jobs=-1,
    random_state=SEED
)
rsf.fit(X_train_full_scaled, y_train_struct)

rsf_train_c = rsf.score(X_train_full_scaled, y_train_struct)
rsf_test_c = rsf.score(X_test_scaled, y_test_struct)

# ================================================================
# 5. [Model 3] DeepSurv (Deep Neural Network)
# ================================================================
y_train_deep = (d_train, e_train)
y_val_deep = (d_val, e_val)
val_data = (X_val_scaled, y_val_deep)

in_features = X_train_scaled.shape[1]
net = tt.practical.MLPVanilla(
    in_features,
    [32, 16],
    out_features=1,
    batch_norm=True,
    dropout=0.2,
    activation=nn.SELU
)

model_deep = CoxPH(net, tt.optim.Adam(lr=0.002, weight_decay=1e-3))
callbacks = [tt.callbacks.EarlyStopping(patience=20)]

log = model_deep.fit(
    X_train_scaled, y_train_deep,
    batch_size=64,
    epochs=200,
    callbacks=callbacks,
    val_data=val_data,
    verbose=False
)
```

```
}

risk_train_deep = model_deep.predict(X_train_scaled).flatten()
risk_test_deep = model_deep.predict(X_test_scaled).flatten()

deepsurv_train_c = concordance_index_censored(e_train.astype(bool), d_train, risk_train_deep)[0]
deepsurv_test_c = concordance_index_censored(e_test.astype(bool), d_test, risk_test_deep)[0]

# ================================================================
# 6. Final Performance Benchmark Table
# ================================================================
summary_data = {
    "Model": ["Cox Proportional Hazards (CPH)", "Random Survival Forest (RSF)", "DeepSurv (Neural Network)"],
    "Model Family": ["Semiparametric / Linear", "Nonlinear Tree Ensemble", "Deep Neural Network"],
    "Train C-Index": [f"{cph_train_c:.4f}", f"{rsf_train_c:.4f}", f"{deepsurv_train_c:.4f}"],
    "Test C-Index": [f"{cph_test_c:.4f}", f"{rsf_test_c:.4f}", f"{deepsurv_test_c:.4f}"]
}

df_results_fair_german = pd.DataFrame(summary_data)


print(f"[German Credit Exact Raw Decomposition] Benchmark")
print(df_results_fair_german.to_string(index=False))
```

```
[German Credit Exact Raw Decomposition] Benchmark
                        Model            Model Family Train C-Index Test C-Index
Cox Proportional Hazards (CPH) Semiparametric / Linear        0.7291       0.7203
  Random Survival Forest (RSF) Nonlinear Tree Ensemble        0.8234       0.7518
     DeepSurv (Neural Network)     Deep Neural Network        0.8112       0.7797
```

F. LLM-as-a-Judge (Cirrhosis Dataset)

```
!pip install google-genai pydantic
```

```
import json
import os
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sksurv.metrics import concordance_index_censored
import torchtuples as tt
from pycox.models import CoxPH

from google import genai
from google.genai import types
from pydantic import BaseModel, Field

from google.colab import userdata
```

```
# ================================================================
# 0. Set random seed for reproducibility
# ================================================================
SEED = 48764388
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# ================================================================
# 1. Feature Engineering and Multivariate Preprocessing
# ================================================================
df_ch_wt = df_ch_dum.copy()

# 1-1. Generate nonlinear Stage-Weighted Risk feature
alpha = np.log(2)
stage_weight = np.exp(alpha * (df_ch_wt["Stage"] - 1.0))
biomarker_risk = (
    np.log(df_ch_wt["Cholesterol"] + 1.0)
    + np.log(df_ch_wt["Tryglicerides"] + 1.0)
    - np.log(df_ch_wt["Albumin" ] + 1e-5)
)
df_ch_wt["Stage_Weighted_Risk"] = stage_weight * biomarker_risk

# 1-2. Define final 10-feature set for Cirrhosis
columns_to_keep = [
    "N_Days", "Status", "Ascites_Y", "Hepatomegaly_Y", "Spiders_Y",
    "Stage_Weighted_Risk", "Bilirubin", "Copper", "Prothrombin",
    "Alk_Phos", "SGOT", "Platelets"
]

df_ch_final = df_ch_wt[[c for c in columns_to_keep if c in df_ch_wt.columns]].copy()

# 1-3. Binarize Status (D: 1.0, C/CL: 0.0) and handle missing values
df_ch_final["Status"] = df_ch_final["Status"].map({"D": 1.0, "C": 0.0, "CL": 0.0})
df_ch_final = df_ch_final.dropna().reset_index(drop=True)
df_ch_final["Status"] = df_ch_final["Status"].astype("float32")
df_ch_final["N_Days"] = df_ch_final["N_Days"].astype("float32")

# ================================================================
# 2. Stratified Data Splitting and Feature Scaling
# ================================================================
feature_cols = [col for col in df_ch_final.columns if col not in ["N_Days", "Status"]]
X = df_ch_final[feature_cols].values.astype("float32")
duations = df_ch_final["N_Days"].values.astype("float32")
events = df_ch_final["Status"].values.astype("float32")

# Step 1: Split into Train+Val (85%) vs Test (15%)
X_train_full, X_test, d_train_full, d_test, e_train_full, e_test = train_test_split(
    X, duations, events, test_size=0.15, random_state=SEED, stratify=events
)

# Step 2: Split Train+Val into Train (70%) vs Val (15%) - For DeepSurv Early Stopping
X_train, X_val, d_train, d_val, e_train, e_val = train_test_split(
    X_train_full, d_train_full, e_train_full, test_size=0.17647, random_state=SEED, stratify=e_train_full
)
```

```
# Standardize features (Fit on Train only to prevent data leakage)
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_train_scaled = scaler.transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

# ==========================================================================
# 3. [Model 3] DeepSurv (Deep Neural Network) Training
# ==========================================================================
y_train_deep = (d_train, e_train)
y_val_deep = (d_val, e_val)
val_data = (X_val_scaled, y_val_deep)

in_features = X_train_scaled.shape[1]
net = tt.practical.MLPVanilla(
    in_features, [32, 16], out_features=1,
    batch_norm=True, dropout=0.2, activation=nn.SELU
)

model_deep = CoxPH(net, tt.optim.Adam(lr=0.002, weight_decay=1e-3))
callbacks = [tt.callbacks.EarlyStopping(patience=20)]

log = model_deep.fit(
    X_train_scaled, y_train_deep,
    batch_size=64, epochs=100, callbacks=callbacks,
    val_data=val_data, verbose=False
)

# Predict risk scores and calibrate risk score direction
risk_train_deep = model_deep.predict(X_train_scaled).flatten()
risk_test_deep = model_deep.predict(X_test_scaled).flatten()

if risk_test_deep[e_test == 1].mean() < risk_test_deep[e_test == 0].mean():
    risk_train_deep = -risk_train_deep
    risk_test_deep = -risk_test_deep

deepsurv_test_c = concordance_index_censored(e_test.astype(bool), d_test, risk_test_deep)[0]
print(f"DeepSurv Test C-Index: {deepsurv_test_c:.4f}")

# ==========================================================================
# 4. LLM-as-a-Judge Integration (Gemini API)
# ==========================================================================
os.environ["GEMINI_API_KEY"] = userdata.get("GEMINI_API_KEY")
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

class ComprehensiveEvaluationResult(BaseModel):
    clinical_interpretability_score: int = Field(description="Score 1-5")
    risk_stratification_score: int = Field(description="Score 1-5")
    actionable_safety_score: int = Field(description="Score 1-5")
    detailed_reasoning: str = Field(description="Comprehensive analysis of the model output.")
    is_safe_for_expert_review: bool = Field(description="Final verdict")

def evaluate_deepsurv_output_with_gemini(patient_id: int, features_dict: dict, predicted_risk: float) -> dict:
    system_instruction = "Evaluate model risk prediction against clinical features."
```

```
    user_content = f"Patient Case: {json.dumps(features_dict)}, Risk: {predicted_risk:.4f}"
    try:
        response = client.models.generate_content(
            model="gemini-3.8-flash",
            contents=user_content,
            config=types.GenerateContentConfig(
                system_instruction=system_instruction,
                response_mime_type="application/json",
                response_schema=ComprehensiveEvaluationResult,
                temperature=0.0
            ),
        )
        return json.loads(response.text)
    except Exception as e:
        return {"error": str(e)}

print("Running LLM-as-a-Judge on Clean Cirrhosis High-Risk Prediction")
highest_risk_idx = np.argmax(risk_test_deep)
sample_features = {col: float(X_test[highest_risk_idx][i]) for i, col in enumerate(feature_cols)}
sample_risk = float(risk_test_deep[highest_risk_idx])

evaluation_result = evaluate_deepsurv_output_with_gemini(
    patient_id=highest_risk_idx,
    features_dict=sample_features,
    predicted_risk=sample_risk
```

```
DeepSurv Test C-Index: 0.7370
Running LLM-as-a-Judge on Clean Cirrhosis High-Risk Prediction
{
    "clinical_interpretability_score": 5,
    "risk_stratification_score": 5,
    "actionable_safety_score": 5,
    "detailed_reasoning": "The patient profile demonstrates acute decompensation and advanced end-stage liver disease, characterized by clinically evident ascites, profound hyperbilirubinemia (17.9 mg/dL), s
    "is_safe_for_expert_review": true
}
```